EX NO:

PATCHPILOT – PYTHON DEBUGGING AND VERIFICATION WORKBENCH

AIM:

To design and implement a web-based debugging workbench that accepts a single Python source file and a bug report, proposes a minimal patch generated by a language model, and verifies that patch by executing the original and the repaired code in a sandbox instead of relying on the model's own claim of success.

ABSTRACT:

PatchPilot is a single-page web application that helps a developer debug one Python file. The user submits the code together with a description of the bug. The server runs the original program in an isolated sandbox, sends the source and the observed output to the Groq language model, and receives a diagnosis, a minimal unified-diff patch and structured regression tests. The patch is applied to a disposable copy, the repaired code is executed again, and the tests are run by a fixed Python runner. A deterministic rule set, not the model, decides whether the repair is verified. If the checks fail, at most two further repair attempts are made. The interface shows the investigation timeline, the code diff in a Monaco editor and the execution console. It is built with Next.js, TypeScript and Tailwind CSS.

OBJECTIVE:

1. To build a single-page workspace containing a code editor, a bug-report form, an investigation timeline, a diff viewer and an execution console.
2. To implement server-side route handlers that validate the input, run the debugging workflow in the background and expose its state through a polling API.
3. To integrate the Groq API and validate its structured JSON output with Zod schemas.
4. To validate unified diffs and apply them only to a disposable, single-file copy of the original source.
5. To execute the original and repaired code through a replaceable sandbox adapter and to run structured regression tests against both versions.
6. To compute the final verification status with deterministic rules and to limit the self-repair loop to two attempts.

TECHNOLOGIES USED:

- **Next.js (App Router) and Node.js** – serves the single-page interface and the Route Handlers (*/api/debug*, */api/debug/[sessionId]*, */api/debug/[sessionId]/retry*, */api/health*) that run the workflow.
- **React and TypeScript (strict mode)** – component-based user interface and typed contracts shared by the client and the server.
- **Tailwind CSS** – utility-based styling of the dark workbench interface; design colours are defined as theme tokens.

- **Monaco Editor (@monaco-editor/react, monaco-editor)** – Python source editor and the side-by-side diff of the original and repaired code; served from the application's own origin.
- **Groq SDK** – calls the configured Groq model for the diagnosis, the patch and the regression tests, using JSON output.
- **Zod** – validates the request body and every field of the model's response before it is used.
- **jsdiff (diff)** – generates the canonical diff and the added and removed line counts for the downloadable patch.
- **Sandbox API (Piston-compatible adapter)** – executes the submitted Python code and the test runner in isolation; the adapter hides the provider behind a TypeScript interface.
- **Python 3 test runner** – a fixed script, executed inside the sandbox, that imports the code under test and compares function results with the expected values read from JSON.
- **lucide-react** – icons used in the interface.
- **Vitest** – unit and integration tests for patching, verdict logic, the sandbox adapter and the workflow.

The project does not use a database. Debugging sessions are kept in server memory and expire after one hour.

PROGRAM:

1. Backend entry point: starting a debugging session (app/api/debug/route.ts)

```
export async function POST(req: Request) {
  let raw: unknown;
  try {
    raw = await req.json();
  } catch {
    return errorResponse(400, "INVALID_JSON", "Request body must be valid JSON.");
  }
  const parsed = debugRequestSchema.safeParse(raw);
  if (!parsed.success) {
    const fields: Record<string, string> = {};
    for (const issue of parsed.error.issues) fields[String(issue.path[0] ?? "request")] ??= issue.message;
    return errorResponse(400, "INVALID_REQUEST", "The request is invalid.", fields);
  }
  const fields = validateRequestFields(parsed.data);
  if (Object.keys(fields).length) return errorResponse(400, "INVALID_REQUEST", "The request is invalid.", fields);

  const cfg = getConfig();
  if (!cfg.groq.configured) {
    return errorResponse(503, "NOT_CONFIGURED", "Groq is not configured. Set GROQ_API_KEY and
GROQ_MODEL on the server.");
  }
  const mode = cfg.sandbox.configured ? "full" : "preview";
  const session = createSession(parsed.data, mode);
  if (!session) return errorResponse(429, "TOO_MANY_SESSIONS", "The server is busy with other investigations. Try
again shortly.");

  // Fire and forget: the client polls GET /api/debug/[id]. Requires a long-lived Node server.
  void runWorkflow(session);
  return json(toView(session), 202);
}
```

2. Interface logic: submitting a bug report and polling its progress (components/Workspace.tsx)

```
const submit = async () => {
  if (submitting.current || running) return;
  const errs = validate(input);
  setErrors(errs);
  if (Object.keys(errs).length) return;
  submitting.current = true;
  setStarting(true);
  setBanner(null);
  setSession(null);
  setTab("orig");
  setSection("timeline");
  try {
    const s = await api.start({
      source: input.source,
      bugReport: input.bugReport.trim(),
      ...(input.expectedBehavior.trim() && { expectedBehavior: input.expectedBehavior.trim() }),
      ...(input.notes.trim() && { notes: input.notes.trim() }),
      ...(input.stdin && { stdin: input.stdin }),
      ...(input.userTests.trim() && { userTests: input.userTests }),
    });
    setSession(s);
    void poll(s.id);
  } catch (e) {
    if (e instanceof ApiRequestError && e.fields) setErrors(e.fields);
    setBanner(e instanceof Error ? e.message : "Could not start the investigation.");
    setSection("input");
  } finally {
    submitting.current = false;
    setStarting(false);
  }
};
```

3. Validated model call with a single re-ask on malformed output (lib/groq.ts)

```
/** Calls the model and validates against a schema; one re-ask on malformed output. */
async function structured<T extends z.ZodType>(schema: T, prompt: string): Promise<z.infer<T>> {
  let lastProblem = "";
  for (let attempt = 0; attempt < 2; attempt++) {
    const text = await complete(
      prompt,
      attempt === 0 ? undefined : `Your previous reply was rejected: ${lastProblem}. Reply again with ONLY a valid
JSON object of the required shape.`,
    );
    try {
      const parsed = schema.safeParse(extractJson(text));
      if (parsed.success) return parsed.data;
      lastProblem = parsed.error.issues
        .slice(0, 4)
        .map((i) => `${i.path.join(".") || "(root)"}: ${i.message}`)
        .join("; ");
    } catch {
      lastProblem = "not valid JSON";
    }
  }
  throw new ModelError("GROQ_INVALID_RESPONSE", `The model did not return a valid structured response
(${lastProblem}).`);
}
```

4. Patch validation: locating each hunk in the analysed source (lib/patch.ts)

```
const eq = (a: string, b: string) => a === b || a.trimEnd() === b.trimEnd();

function findHunk(orig: string[], old: string[], hint: number, minStart: number): number | string {
  const matchesAt = (i: number) => {
    if (i < minStart || i + old.length > orig.length) return false;
    for (let k = 0; k < old.length; k++) if (!eq(orig[i + k]!, old[k]!)) return false;
    return true;
  };
  if (old.length === 0) return "Hunk has no context or removed lines to anchor it.";
  if (matchesAt(hint)) return hint;
  const found: number[] = [];
  for (let i = minStart; i + old.length <= orig.length; i++) if (matchesAt(i)) found.push(i);
  if (found.length === 1) return found[0]!;
  if (found.length === 0) return "Hunk context does not match the submitted source (the patch was not made against this
exact version).";
  return "Hunk context matches several locations; it is ambiguous.";
}
```

5. Deterministic check that decides whether a repair attempt failed (lib/verdict.ts)

```
export const failed = (t?: TestRun) => !!t?.outcomes.some((o) => o.status === "failed" || o.status === "error");
export const allPassed = (t?: TestRun) => !!t && t.ran && t.outcomes.length > 0 && t.outcomes.every((o) => o.status
=== "passed");
const crashed = (e?: ExecutionResult) => !!e && !e.providerError && (e.timedOut || (e.exitCode !== null &&
e.exitCode !== 0));
const clean = (e?: ExecutionResult) => !!e && !e.providerError && !e.timedOut && e.exitCode === 0;

/** True when this attempt applied and then failed some executed check — the trigger for a repair attempt. */
export function attemptFailedChecks(a: Attempt): boolean {
  if (a.patchStatus !== "applied") return false;
  if (a.repairedExecution && !a.repairedExecution.providerError && !clean(a.repairedExecution)) return true;
  if (failed(a.generatedTests)) return true;
  if (a.userTests?.ran && !a.userTests.passed) return true;
  return false;
}
```

OUTPUT:

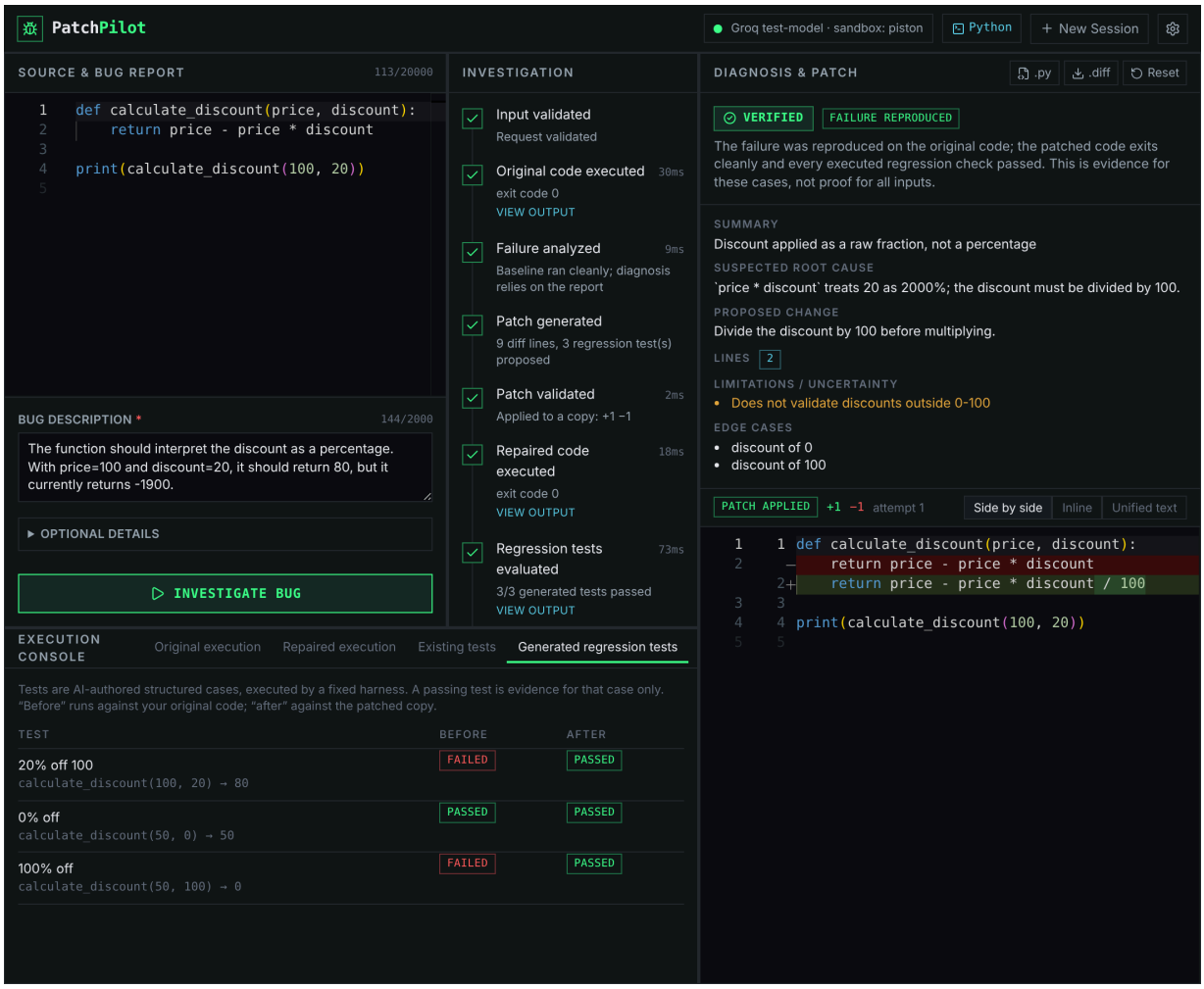


Fig. 1 PatchPilot workspace after an investigation

RESULT:

Thus, the program is executed and the output is verified. The original code returned -1900; after the patch it returned 80 and all 3 regression tests passed (status: VERIFIED).